

Day 10 - Asynchronous JavaScript: Callbacks, the Event Loop, Promises, async/await

Day 10 — Asynchronous JavaScript: Callbacks, the Event Loop, Promises, async/await

Objectives

- Understand *why* JavaScript needs a special way of handling things that take time (this is arguably the single most defining feature of the language, and the topic most beginners find hardest)
- Understand callbacks, and the problem with them ("callback hell") that motivated everything else in this lesson
- Understand Promises — the modern building block for asynchronous code
- Understand `async/await` — the clean, modern syntax built on top of Promises
- Understand basic error handling in asynchronous code

Why does JavaScript need "asynchronous" code at all?

JavaScript, in both the browser and in Node, runs on a **single thread** — meaning it can only actually execute one single instruction at a time, ever (unlike some languages that can genuinely run multiple things simultaneously). Now imagine your program needs to read a large file from disk, or fetch data from a server somewhere across the internet — these things can take a noticeable amount of time (milliseconds to seconds). If JavaScript simply *paused* everything and waited, your entire program (in a browser: the entire web page, completely unresponsive to clicks or scrolling) would freeze solid until that slow operation finished.

Asynchronous ("async" for short) programming is JavaScript's answer: instead of pausing everything to wait, you say "start this slow operation, and let me know when it's done — meanwhile, keep running everything else." The rest of your program keeps working normally, and your "let me know when it's done" code runs later, once the slow thing actually finishes.

The event loop — the mechanism that makes this possible

JavaScript's runtime (Node, or a browser) uses something called the **event loop** to manage this. In simplified terms: JavaScript keeps a queue of "things to do once something slow finishes." It runs your normal, synchronous code (top to bottom, one instruction at a time, exactly like everything you've written so far this week) until there's nothing left to do *right now* — then it checks that queue, and if something on it is ready (like a network request that just finished), it runs the corresponding "let me know when it's done" code. Then it checks the queue again. This cycle — run what's ready, check the queue, repeat — is the "loop" in "event loop." You don't need to manage this cycle yourself; you just need to understand that it exists, since it explains some genuinely surprising ordering behavior you'll see in the exercises below.

Callbacks — the original, older approach

The earliest way JavaScript handled "let me know when it's done" was a **callback**: a function you hand to the slow operation, which it calls for you once it's finished:

```
const fs = require("fs");

fs.readFile("data.txt", "utf-8", (error, contents) => {
  if (error) {
    console.log("Something went wrong:", error.message);
    return;
  }
  console.log(contents);
});

console.log("This line runs BEFORE the file's contents are printed!");
```

Read this carefully: `fs.readFile` starts reading the file and immediately returns, WITHOUT waiting — the callback function `((error, contents) => { ... })` is only actually called later, once the file has genuinely finished being read. This is why `"This line runs BEFORE..."` prints **before** the file's contents, even though it's written **after** the `readFile` call in your source code — this is the entire point of asynchronous code: your program doesn't stop and wait.

The problem: "callback hell"

Callbacks work, but they get unwieldy fast once you need to do several async things in sequence, where each one depends on the previous one finishing first:

```
getUser(userId, (user) => {
  getOrders(user.id, (orders) => {
    getOrderDetails(orders[0].id, (details) => {
      console.log(details);
      // and this nesting just keeps growing sideways with each additional step...
    });
  });
});
```

This deeply-nested, sideways-growing shape is genuinely known in the JavaScript community as **"callback hell"** — it's hard to read, hard to add proper error handling to at every level, and hard to modify safely. This exact problem is what motivated everything else in today's lesson.

Promises — a value representing "the result of something that hasn't finished yet"

A **Promise** is an object representing an operation that hasn't completed yet, but will (eventually) either **resolve** (succeed, with some resulting value) or **reject** (fail, with some error). Instead of nesting callbacks, you chain `.then()` (for success) and `.catch()` (for failure):

```
function delay(ms) {
  return new Promise((resolve) => {
    setTimeout(() => resolve(`Waited ${ms}ms`), ms);
  });
}

delay(1000)
  .then((message) => {
```

```

        console.log(message);          // "Waited 1000ms" -- runs AFTER 1000ms have passed
        return delay(500);             // returning a Promise from .then() lets you CHAIN to the next
    })
    .then((message) => {
        console.log(message);          // "Waited 500ms"
    })
    .catch((error) => {
        console.log("Something failed:", error.message); // runs if ANY step above rejected
    });

```

This flattens what would have been deeply-nested callbacks into a readable, top-to-bottom **chain** — each `.then()` runs only after the previous step finished successfully, and a single `.catch()` at the end can handle a failure from *any* step in the chain, rather than needing separate error-handling at every nested level.

`setTimeout(callback, ms)` (used inside `delay` above) is a built-in function that runs `callback` after roughly `ms` milliseconds have passed, without blocking anything else — it's one of the most basic building blocks of asynchronous JavaScript, and a handy way to simulate "something slow" in your own practice code, exactly as done here.

`async`/`await` — the modern, clean syntax built on top of Promises

`async/await` doesn't replace Promises — it's a way of *writing* Promise-based code that reads almost exactly like ordinary, synchronous, top-to-bottom code, hiding most of the `.then()` chaining:

```

async function run() {
    const message1 = await delay(1000); // PAUSES this function here until the Promise resolves, v
    console.log(message1);              // "Waited 1000ms"

    const message2 = await delay(500);
    console.log(message2);              // "Waited 500ms"
}

run();

```

Two rules to know: `await` **can only be used inside a function marked `async`** (you'll get a `SyntaxError` otherwise), and `await` **pauses only that one function**, waiting for the Promise to settle, while the rest of your program (anything not inside that specific paused function) keeps running completely normally in the meantime — this is the same underlying async behavior as the callback example above, just written in a much more readable, linear style.

Error handling with `async`/`await` — back to `try`/`catch`

Because `async/await` makes asynchronous code look synchronous, you can go back to using ordinary `try/catch` (Day 6) to handle failures, instead of `.catch()`:

```

async function run() {
    try {
        const message = await delay(1000);
        console.log(message);
    } catch (error) {
        console.log("Something failed:", error.message);
    }
}

```

This is generally considered the clearest, most maintainable way to write asynchronous JavaScript today, and it's what you should default to for any new code — you should still recognize raw `.then().catch()` chains when you see them in existing code, and understand Promises well enough to know what `async/await` is actually doing underneath, but prefer `async/await` for anything you write yourself from here on.

Running several async operations at once with `Promise.all`

If you have several independent async operations that don't depend on each other, awaiting them one at a time wastes time waiting for each to finish before even starting the next. `Promise.all` lets you start them all simultaneously and wait for every one to finish:

```
async function run() {
  const [message1, message2] = await Promise.all([delay(1000), delay(500)]);
  console.log(message1, message2); // both ready together, after roughly 1000ms total (the LONGER)
}
```

This is a genuinely important performance pattern: if three independent network requests each take about a second, awaiting them one after another takes roughly three seconds total; `Promise.all`-ing them takes roughly one second total, since they all run concurrently.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`. Since this involves real timing, some tests may take a second or two to finish — that's expected.